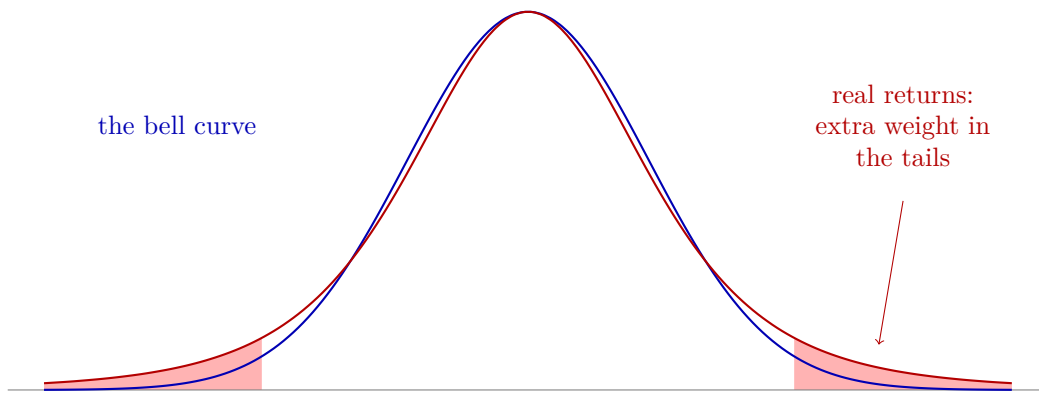


A FRIENDLY GUIDE TO

Guideline for Noobs

Quant finance, risk and the EUR/USD project
explained like you are curious, not like you are an expert



Hi, I'm Hugo. I like statistics, probability and programming, but I am *not* an advanced quant. I wrote this guide to learn, so I explain every idea the way I wish someone had explained it to me: small examples first, formulas second. If something here is unclear, that is a bug in the guide, not in you.

Hugo Johansson

October 2026

Declaration of AI use. The author of this guide is Hugo Johansson, who takes full responsibility for the content and the final version. The code, the guide and the literature search were produced with Claude Sonnet 5.5 (Anthropic) and a number of Claude sub-agents (a research agent for the sources and a writing agent for the guide). The code and the guide were afterwards reviewed by two other AI systems, Gemini (Google) and ChatGPT (OpenAI). AI review does not replace checking by the author; see `AI_USE.md` in the repository for a task-by-task log.

This guide is for learning only. It is not investment advice, and nothing in it says where EUR/USD is going.

Contents

1	How to use this guide	3
2	Part A: Glossary (begreppslista)	3
A1.	Prices, returns and the EUR/USD quote	4
2.1	EUR/USD quote convention	4
2.2	ECB reference rate	4
2.3	Price vs return	4
2.4	Log return	5
2.5	Mean	5
2.6	Standard deviation	5
2.7	Volatility	6
A2.	Distributions and their shape	6
2.8	Normal distribution	6
2.9	Skewness	6
2.10	Kurtosis and excess kurtosis	7
2.11	Fat tails	7
2.12	QQ plot	7
A3.	Testing ideas with data	8
2.13	Hypothesis test and p-value	8
2.14	Jarque–Bera test	8
A4.	Value-at-Risk	9
2.15	Value-at-Risk (VaR)	9
2.16	Confidence level	9
2.17	Parametric (normal) VaR	9
2.18	Historical VaR (historical simulation)	10
2.19	Monte Carlo VaR	10
A5.	Checking a VaR model	10
2.20	Exceedance	10
2.21	Backtesting	10
2.22	Rolling window	11
2.23	Look-ahead bias	11

2.24	Kupiec test	11
A6.	Simulation	12
2.25	Monte Carlo	12
2.26	Bootstrap	12
2.27	Scenario analysis vs forecast	12
A7.	Volatility that changes over time	13
2.28	Volatility clustering	13
2.29	Conditional heteroscedasticity	13
2.30	ARCH	13
2.31	GARCH-effect	13
A8.	Exchange rates	14
2.32	Random walk	14
3	Part B: The project step by step	14
3.1	Step 1: Data and log returns	14
3.2	Step 2: Are the returns normal? (Jarque–Bera)	15
3.3	Step 3: Three ways to compute VaR	16
3.4	Step 4: The rolling backtest without look-ahead	17
3.5	Step 5: Counting exceedances and the Kupiec test	17
3.6	Step 6: Scenario analysis to 2040 (bootstrap)	18
4	Part C: How currency exchange rates vary with macroeconomic trends	20
4.1	Interest rate differentials and interest rate parity	20
4.2	Inflation and purchasing power parity (PPP)	20
4.3	Central bank policy: ECB vs Fed	21
4.4	Dornbusch overshooting	21
4.5	Safe-haven demand for the US dollar	22
4.6	The honest message: models struggle to beat a random walk	22
5	Limitations	23
6	Exercises	23
7	Cheat sheet	25

1 How to use this guide

This guide explains the words and ideas behind my GitHub project *fx-var-normality-test*. The project asks two questions:

1. Are daily EUR/USD returns normally distributed?
2. If we pretend they are, do we underestimate the risk?

The short answers are “no” and “yes, at the 99% level”. The rest of this guide explains what those answers mean.

Who is this for? A first-year student who knows some basic Python and has seen a Monte Carlo simulation, but has not studied quant finance. If you can write a `for` loop and you know what an average is, you are ready.

How is it built?

- **Part A** (Section 2) is a glossary (*begreppslista*). Every word gets a plain definition, a tiny example with numbers, and a note on where it shows up in the project. Use it like a dictionary. You do not have to read it from top to bottom.
- **Part B** (Section 3) walks through the project step by step, with the real results.
- **Part C** (Section 4) is about how exchange rates move with the economy, told as a made-up story where we trade EUR/USD.
- Then come limitations, exercises with answers, a one-page cheat sheet of formulas, and the references.

The boxes. Three kinds of boxes appear again and again:

In one sentence

The shortest possible version of an idea. If you only read these, you still get the main points.

Example

A small calculation with easy numbers, so you can check it on paper or in Python.

Common mistake

A trap that beginners (including me) fall into.

A rule for the whole guide. When I say something about what researchers have found, there is a reference in brackets, like (Jorion, 2007). When there is no reference, it is either a definition, a calculation you can check yourself, or my own reasoning, and I try to say which. Numbers from the project come from running `var_eurusd.py` on ECB data from 4 January 1999 to 7 October 2026.

2 Part A: Glossary (*begreppslista*)

The terms are grouped so that each one only uses words that came before it.

A1. Prices, returns and the EUR/USD quote

2.1 EUR/USD quote convention

In plain words. “EUR/USD = 1.1177” means *one euro costs 1.1177 US dollars*. The euro is the thing being priced; the dollar is the money we pay with. So when the number goes *up*, the euro has become *stronger* (you need more dollars to buy one euro). A rate going up gives a *positive* return.

Example

Monday EUR/USD is 1.10, Tuesday it is 1.12. One euro now buys 2 more US cents. The euro got stronger, the dollar got weaker, and the return is positive.

In the project. The ECB series D.USD.EUR.SP00.A gives how many dollars one euro costs. The last value in the data is 1.1177. All returns in the project are from the point of view of someone who *holds euros and counts their money in dollars*. For that person, a negative return is a loss.

Common mistake

Mixing up the direction. “EUR/USD fell” means the *euro* fell, not the dollar. If you hold dollars and count in euros, a fall in EUR/USD is good news for you.

2.2 ECB reference rate

In plain words. Every working day the European Central Bank publishes one official EUR/USD number. It is a snapshot taken in the early afternoon (the ECB’s page says around 14:10 CET; the data file itself labels it 2.15 pm CET) and published around 16:00 CET (European Central Bank, 2026). The ECB says the rates are for information, not for doing trades (European Central Bank, 2026).

Tiny example. On a given day the market price moves thousands of times, but the ECB data file contains just one number for that day, for example 1.1177.

In the project. The script downloads this series. Because it is one number per day, we cannot see moves inside the day, and there is no bid/ask spread (the small gap between buying and selling price a bank would charge you).

2.3 Price vs return

In plain words. A *price* is a level (“1.10 dollars per euro”). A *return* is a change, in percent (“up 2% since yesterday”). Risk models work with returns, because a 1-cent move means very different things if the price is 1 dollar or 100 dollars.

Example

A price goes from 100 to 110. The simple return is $(110 - 100)/100 = 10\%$. A price going from 1.00 to 1.10 also has a 10% return, even though it moved only 0.10.

In the project. The data contains prices (rates). The first thing the script does is turn them into returns: 7109 prices give 7108 returns.

2.4 Log return

In plain words. A log return is a different way to measure the change:

$$r_t = 100 \cdot \ln\left(\frac{P_t}{P_{t-1}}\right) \quad (\text{in percent}).$$

For small changes it is almost the same as the simple return. Its big advantage: log returns *add up*. The log return over two days is the sum of the two daily log returns.

Example

Prices: 100, 110, 99.

- Day 1: $100 \ln(110/100) = +9.53\%$.
- Day 2: $100 \ln(99/110) = -10.54\%$.
- Sum: -1.01% , which is exactly $100 \ln(99/100)$, the total change.

With simple returns you get $+10\%$ and -10% . They add up to 0% , but the price actually fell from 100 to 99. Simple returns do not add up; log returns do.

In the project. `log_returns(rates)` computes `100 * np.diff(np.log(rates))`. The 2040 scenario uses the adding-up property: it adds daily log returns and then uses `np.exp` to get back to a price.

Common mistake

Forgetting the factor 100. Then a “1.3” VaR would be read as 1.3 (that is 130%) instead of 1.3%.

2.5 Mean

In plain words. The average. Add everything up and divide by how many numbers there are. Written μ (“mu”).

Tiny example. Returns $+1, -1, +2, -2, +5$: $\text{mean} = 5/5 = 1\%$.

In the project. The mean daily EUR/USD log return over 1999–2026 is about -0.0008% . That is basically zero. Over one day, the direction is close to a coin flip, and the size of the moves matters much more than the average.

2.6 Standard deviation

In plain words. How far the numbers typically are from the mean. Written σ (“sigma”). Steps: subtract the mean, square, average, take the square root.

$$\sigma = \sqrt{\frac{1}{n} \sum_{t=1}^n (r_t - \mu)^2}.$$

Example

Returns 1, -1, 2, -2. Mean = 0. Squares: 1, 1, 4, 4. Average of squares = $10/4 = 2.5$. Square root: $\sigma \approx 1.58\%$.

In the project. The daily standard deviation is about 0.58%. Note that `np.std` divides by n , not $n - 1$. With 500 or 7108 numbers the difference is tiny.

2.7 Volatility

In plain words. In finance, “volatility” usually means the standard deviation of returns. High volatility = big swings. Think of it as how “stormy” the market is. Careful: there are several versions. *Sample (historical) volatility* is the standard deviation of past returns, as here. *Annualised volatility* rescales it to one year. *Conditional volatility* is the volatility on one particular day given what happened before, and it can change from day to day (GARCH models, below). A *forecast volatility* is a prediction of the future.

Tiny example. Daily volatility 0.58% is a standard deviation: moves of roughly that size are common. (It is not the same as the average absolute move, which is a bit smaller for a bell curve, about 0.8σ .) A rough yearly number is $0.58\% \times \sqrt{252} \approx 9\%$ (this square-root-of-time rule assumes independent days).

In the project. Volatility is the σ in the normal VaR formula, estimated from the last 500 days.

A2. Distributions and their shape

2.8 Normal distribution

In plain words. The famous bell curve. It is completely described by two numbers: the mean μ and the standard deviation σ . About 68% of values are within 1σ of the mean, about 95% within 2σ , and almost all (99.7%) within 3σ . Very large values are *extremely* rare.

Example

Heights of adults are close to normal. If the mean is 175 cm and $\sigma = 7$ cm, someone 196 cm tall is 3σ above the mean: rare but you have seen it. Someone 8σ tall (231 cm) basically never exists.

In the project. The normal VaR and the Monte Carlo VaR assume that returns are normal. The project tests if that is true. (Spoiler: no.)

2.9 Skewness

In plain words. Is the distribution lopsided? Skewness 0 means symmetric, like the normal. Negative skew means a longer tail of big *losses*; positive skew means a longer tail of big *gains*.

Tiny example. A lottery ticket: you usually lose a little, and very rarely win a lot. That is strong positive skew.

In the project. The skewness of EUR/USD returns is about 0.05, very close to zero. So the up moves and down moves are roughly mirror images. The problem with normality is not the skew, it is the kurtosis.

2.10 Kurtosis and excess kurtosis

In plain words. Kurtosis measures how much of the action is in the *tails* (the rare, extreme values). A normal distribution has kurtosis 3. Because everyone compares with the normal, people use *excess* kurtosis = kurtosis – 3. So a normal has excess kurtosis 0.

Example

Picture two dice games that both have the same average and the same standard deviation. Game A is mostly middle results. Game B is mostly very small moves, but now and then a huge one. Game B has higher kurtosis: quiet most of the time, with sudden shocks.

In the project. `stats.kurtosis` in SciPy returns *excess* kurtosis by default. For EUR/USD it is 3.31, so the kurtosis is about 6.31. That is clearly more than a normal.

Common mistake

Reading SciPy’s “3.31” as “kurtosis is about 3, so it is normal”. SciPy already subtracted 3. The normal value is 0, not 3.

2.11 Fat tails

In plain words. “Fat tails” (or heavy tails) means extreme values happen much more often than the normal distribution says. Financial returns are well known for this (Mandelbrot, 1963; Fama, 1965; Cont, 2001), and daily exchange rates too (Baillie and Bollerslev, 1989).

Example

The worst daily EUR/USD log return in the data is about -4.7% (19 December 2008). With $\sigma = 0.58\%$ that is about 8σ . Under a normal distribution the chance of a day that bad is about 2×10^{-16} , less than one in a trillion. We have only about 7100 days of data, yet it happened. One extreme day alone does not prove fat tails, but together with the excess kurtosis, the QQ plot and the Jarque–Bera test below, the evidence against a thin-tailed normal model is strong.

In the project. Fat tails are why the normal VaR at 99% is too low.

2.12 QQ plot

In plain words. A picture that compares your data with a normal distribution. Sort your returns from smallest to largest. Next to each one, put the value a normal distribution would “expect” at that rank. Plot one against the other. If the data are normal, the points lie on a straight line. If the ends bend away from the line, the tails are fatter (or thinner) than normal.

Tiny example. If you have 100 returns, the 1st smallest should be near the normal’s 1% point (about $\mu - 2.33\sigma$). If your smallest return is instead $\mu - 4\sigma$, that point sits far below the line.

In the project. Figure 2 (`figures/qq_plot.png`), made with `stats.probplot`. The middle follows the line, the two ends bend away: an S-shape, the classic sign of fat tails.

A3. Testing ideas with data

2.13 Hypothesis test and p-value

In plain words. A hypothesis test is a fair way to ask “could this just be bad luck?”. You start with a boring assumption, the *null hypothesis* H_0 (“the coin is fair”, “returns are normal”). Then you compute how surprising your data would be *if* H_0 were true. That surprise level is the p-value: the probability of getting data at least this extreme when H_0 is true. A small p-value (usual cutoff: 0.05) means “too surprising, I reject H_0 ”.

Example

You flip a coin 10 times and get 10 heads. If the coin is fair, the chance of that is $0.5^{10} \approx 0.001$. That p-value is tiny, so you reject “the coin is fair”. If you got 6 heads, the p-value would be large and you would not reject.

In the project. Two tests use p-values: Jarque–Bera (are returns normal?) and Kupiec (does a VaR model break as often as it should?).

Common mistake

Saying “p = 0.34, so the model is proven correct”. A large p-value only means “no evidence against it”. It does not prove H_0 is true.

2.14 Jarque–Bera test

In plain words. A test of “the data are normal” that uses two numbers: skewness S and excess kurtosis K . Both are 0 for a normal, so if either is far from 0 the statistic becomes large (Jarque and Bera, 1980, 1987):

$$JB = \frac{n}{6} \left(S^2 + \frac{K^2}{4} \right) \sim \chi^2(2) \quad \text{if } H_0 \text{ is true (approximately, for large } n).$$

The 5% critical value of $\chi^2(2)$ is about 5.99. The $\chi^2(2)$ result is an approximation for large n , and it treats the observations as independent. Daily returns show volatility clustering (below), so the exact p-value should be taken with care. With a statistic this large, the conclusion does not change.

Example

Suppose $n = 600$, $S = 0$ and $K = 0.4$. Then $JB = 100 \cdot (0 + 0.16/4) = 4$. That is below 5.99, so we do not reject normality. If instead $K = 1$, then $JB = 100 \cdot 0.25 = 25$, and we reject.

In the project. With $n = 7108$, $S = 0.05$ and $K = 3.31$: $JB = \frac{7108}{6}(0.0025 + 2.74) \approx 3244$. The p-value is basically 0. Normality is clearly rejected.

Common mistake

Thinking that a big n makes the test “unfair”. It is true that with many days even a small deviation from normal is detected. But here the deviation is not small: an excess kurtosis of 3.31 is large.

A4. Value-at-Risk

2.15 Value-at-Risk (VaR)

In plain words. A VaR answers: “On a normal bad day, how much could I lose?” More exactly: the 99% one-day VaR is a loss level that you expect to be beaten on only 1% of days (Jorion, 2007). In the project it is written as a *positive* percentage loss.

Example

“The 99% one-day VaR is 1.3%.” This means: on 99 days out of 100 you should lose less than 1.3%. On about 1 day in 100 (two or three days a year) you lose more. If you hold euros worth 1,000,000 dollars, a 1.3% loss is 13,000 dollars.

In the project. Every day of the backtest, three methods compute a VaR at 95% and 99%.

Common mistake

Thinking VaR is the *worst* possible loss. It is not. VaR says nothing about *how big* the loss is on the 1% bad days. That is a known weakness; the measure “expected shortfall” looks at the average loss in the tail instead (Artzner et al., 1999).

2.16 Confidence level

In plain words. The “99%” or “95%” in VaR. It says how often the VaR line should hold. 99% means “broken 1 day in 100”, 95% means “broken 1 day in 20”. A higher level means a line further out in the tail, so a bigger VaR number.

Tiny example. It is like a flood wall. A wall built for the “1 in 20 years” flood is lower than a wall built for the “1 in 100 years” flood.

In the project. LEVELS = [0.95, 0.99]. The interesting level is 99%, because that is deep in the tail, where fat tails matter most.

2.17 Parametric (normal) VaR

In plain words. Assume returns are normal, estimate μ and σ , and read the cutoff from the bell curve:

$$\text{VaR} = -(\mu + z\sigma), \quad z = \Phi^{-1}(1 - \text{level}),$$

where Φ^{-1} is the inverse of the normal distribution function (`stats.norm.ppf`). At 99%, $z = \Phi^{-1}(0.01) \approx -2.326$; at 95%, $z \approx -1.645$. Because z is negative, $\mu + z\sigma$ is a negative return (a loss), and the minus sign in front makes VaR positive. This normal (“variance–covariance”) approach is the one described in the RiskMetrics technical document (J.P. Morgan/Reuters, 1996).

Example

Using the whole sample: $\mu = -0.0008\%$ and $\sigma = 0.581\%$. 99% VaR = $-(-0.0008 + (-2.326)(0.581)) \approx 1.35\%$.

In the project. `var_normal(window, level)`. Over the backtest, the average 99% normal VaR is 1.32%.

2.18 Historical VaR (historical simulation)

In plain words. Do not assume any shape. Take your past returns, sort them, and pick the one that is 1% from the bottom (for 99%). “The past is my distribution.” Its weaknesses are discussed by Pritsker (2006).

Example

You have 500 past returns. 1% of 500 is 5. So the 99% VaR is roughly the 5th or 6th worst return (`np.quantile` interpolates between neighbouring values). If the 5 worst days were $-2.1, -1.8, -1.6, -1.5, -1.45\%$, the VaR would be about 1.45%.

In the project. `var_historical` uses `-np.quantile(window, 1 - level)`. Average 99% historical VaR in the backtest: 1.41%, higher than the normal one, because the real tail is fatter than the bell curve.

2.19 Monte Carlo VaR

In plain words. Simulate many fake returns from a model, then use the historical method on the fake returns. In the project the model is a normal distribution with the window’s μ and σ .

Tiny example. Draw 10,000 numbers from a normal with $\mu = 0, \sigma = 0.58$. The 100th smallest is close to $-2.326 \times 0.58 \approx -1.35$. So the simulated VaR is close to the formula VaR.

In the project. `var_monte_carlo` with `N_DRAWS = 10000`. Because it uses the same normal assumption, it should land very close to the parametric normal VaR, with a little simulation noise. In this particular run the exceedance counts happen to be identical (114 and 302). Monte Carlo is only as good as the model you feed it.

Common mistake

Thinking “Monte Carlo” automatically means “more realistic”. A simulation from a normal distribution still has thin normal tails.

A5. Checking a VaR model

2.20 Exceedance

In plain words. A day when the loss is bigger than the VaR. Also called a “hit”, “breach” or “violation”. In the project: $r_t < -\text{VaR}_t$.

Tiny example. The VaR is like a speed limit and an exceedance is a speeding ticket. A 99% VaR should give about 1 ticket per 100 days. If you get 2 tickets every 100 days, the limit is set wrong.

In the project. `n_exceed = int(np.sum(test_returns < -var_values))`.

2.21 Backtesting

In plain words. Pretend you are back in the past. Every day, compute the VaR using only what you knew then, and see whether the next return broke it. Then count the exceedances. Banking regulators use this idea to check banks’ VaR models (Basel Committee on Banking Supervision, 1996).

Tiny example. With 1000 test days at 99%, you expect about 10 exceedances. If you see 25, the model is too optimistic. If you see 1, it is too cautious.

In the project. 6608 test days. At 99% we expect $0.01 \times 6608 = 66.1$ exceedances.

2.22 Rolling window

In plain words. Use only the last W days to estimate the model, and move the window forward one day at a time. Old data drop out, new data come in.

Tiny example. $W = 500$. To estimate the VaR for day 501, use days 1–500. For day 502, use days 2–501. And so on.

In the project. `WINDOW = 500` (about two years of working days), and `window = returns[day - WINDOW:day]`.

2.23 Look-ahead bias

In plain words. Cheating by accident: using information that you could not have known at the time. It makes a backtest look better than reality.

Example

You test a VaR for 15 September 2008, but your window includes data from October 2008. Your model “knows” the crisis is coming. In real life it could not.

In the project. The Python slice `returns[day - WINDOW:day]` stops *before* `day` (the end of a slice is not included). So the VaR for day `day` only uses earlier returns.

Common mistake

Writing `returns[day - WINDOW:day + 1]`. One extra day sounds harmless, but it puts the very return you are testing into the window.

2.24 Kupiec test

In plain words. A test of “the model breaks exactly as often as it promises” (Kupiec, 1995). With n test days, x exceedances and a target rate p (0.01 at 99%), the test compares two likelihoods: how likely x is if the true rate is p , and how likely it is if the true rate is the observed x/n :

$$LR = -2 \ln \left[\frac{(1-p)^{n-x} p^x}{(1-\frac{x}{n})^{n-x} (\frac{x}{n})^x} \right] \sim \chi^2(1) \quad \text{if } H_0 \text{ is true.}$$

The 5% critical value of $\chi^2(1)$ is about 3.84.

Example

One year: $n = 250$ days, $p = 0.01$, so we expect 2.5 exceedances. We see $x = 6$. Plugging in gives $LR \approx 3.56$ and a p-value of about 0.06. Just above 0.05, so we (barely) do not reject. More than double the expected number of breaks, and still not rejected: one year at 99% is too short to say much.

In the project. `kupiec_test(n_days, n_exceed, p)`. The code works with logs, so it adds $x \ln p$ instead of multiplying p^x , which avoids numbers too small for the computer.

Common mistake

Thinking “few exceedances” is always good. Too *few* is also a failure: the VaR is too high, and a bank would hold more capital than needed. Kupiec rejects in both directions. Kupiec also does not check whether exceedances come in clusters; that needs another test (Christoffersen, 1998).

A6. Simulation

2.25 Monte Carlo

In plain words. Answer a question by simulating it many times with random numbers and looking at the results (Glasserman, 2003). Named after the casino.

Tiny example. What is the chance that two dice sum to 7? Roll them 100,000 times in Python and count. You get close to $6/36 \approx 16.7\%$.

In the project. Used twice: the Monte Carlo VaR (normal draws) and the 2040 scenarios (bootstrap draws). Both use `np.random.seed(42)`, so you get the same numbers every run.

2.26 Bootstrap

In plain words. A Monte Carlo method where you do not invent a distribution. Instead you draw *from your own data*, with replacement (Efron, 1979; Efron and Tibshirani, 1986).

Example

Write each of the 7108 historical daily returns on a marble and put them all in a bag. To simulate one future day, pull out a marble, write down the number, and put it back. Repeat for every future day. Because the real crash days are in the bag, the fat tails come along automatically.

In the project. The function `scenario_to_2040` does this with `np.random.choice(returns, size=(n_paths, n_days))`.

2.27 Scenario analysis vs forecast

In plain words. A *forecast* says “this is where I think it will be”. A *scenario analysis* says “if the future behaves like this, here is the range of things that could happen”. It is a map of possibilities, not a prediction.

Tiny example. “Tomorrow will be 14 degrees” is a forecast. “In Stockholm in October, the temperature is usually between 2 and 14 degrees” describes a range of scenarios. The second is useful for packing a suitcase, but it does not tell you what tomorrow will be.

In the project. The fan chart to 2040 is a scenario analysis. Why I do not call it a forecast is my own reasoning, explained in Part B: the model has no idea about direction (the mean is about zero), and research shows that exchange rates are very hard to forecast at all (Meese and Rogoff, 1983; Rossi, 2013).

A7. Volatility that changes over time

2.28 Volatility clustering

In plain words. Calm days tend to follow calm days, and wild days tend to follow wild days. Volatility comes in “spells”. It is one of the best-known facts about financial returns (Cont, 2001).

Tiny example. Like weather: if today is stormy, tomorrow is more likely to be stormy than a random day in the year. Storms come in periods, not one isolated day at a time.

In the project. You can see it in Figure 3: the grey returns are much wider in 2008–2009 than in quiet years. The project does *not* model this. A 500-day window reacts slowly, and the bootstrap draws days independently.

2.29 Conditional heteroscedasticity

In plain words. A long name for a simple idea. “Heteroscedasticity” = the variance is not constant. “Conditional” = the variance today depends on what happened before (it is conditional on the past). Volatility clustering is the pattern you can see in a chart, and it is consistent with a conditional variance that changes over time.

Tiny example. If yesterday’s move was -2% , our best guess for today’s volatility is higher than if yesterday’s move was $+0.1\%$, even though we have no idea about today’s direction.

In the project. Not modelled. The normal VaR uses one σ for the whole 500-day window, as if the variance were constant inside it.

2.30 ARCH

In plain words. “AutoRegressive Conditional Heteroscedasticity” (Engle, 1982). Today’s variance is a constant plus a part of yesterday’s *squared* surprise ε_{t-1} (the return minus its mean):

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2.$$

Squared, because a big move *in either direction* raises volatility.

Tiny example. With $\omega = 0.2$ and $\alpha = 0.3$ (made-up numbers): after a 2% move, $\sigma_t^2 = 0.2 + 0.3 \cdot 4 = 1.4$, so $\sigma_t \approx 1.18\%$. After a 0.1% move, $\sigma_t^2 = 0.2 + 0.003 \approx 0.2$, so $\sigma_t \approx 0.45\%$.

In the project. Not used; listed under “Possible extensions” in the README.

2.31 GARCH-effect

Volatility clustering + conditional heteroscedasticity. After a volatile movement, a period of volatility follows, and vice versa: after a calm period, more calm follows. (More standard names for this: “volatility clustering”, “volatility persistence”, or simply “a GARCH model”).

In plain words. GARCH (“Generalized ARCH”) adds one more piece to ARCH: yesterday’s *variance* itself (Bollerslev, 1986). The most used version is GARCH(1,1):

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2.$$

The β term is the “memory”: high volatility yesterday gives high volatility today, which gives high volatility tomorrow, and it fades only slowly. That is exactly the clustering in the entry

above. GARCH models are used widely in finance (Bollerslev et al., 1992), including for daily exchange rates (Baillie and Bollerslev, 1989). A well-known comparison study asks whether other volatility models beat GARCH(1,1) (Hansen and Lunde, 2005).

Example

Made-up numbers: $\omega = 0.01$, $\alpha = 0.05$, $\beta = 0.90$, and yesterday $\sigma_{t-1} = 0.6\%$.

- Yesterday's surprise was -2% : $\sigma_t^2 = 0.01 + 0.05 \cdot 4 + 0.90 \cdot 0.36 = 0.534$, so $\sigma_t \approx 0.73\%$. Volatility goes up.
- Yesterday's surprise was $+0.1\%$: $\sigma_t^2 = 0.01 + 0.0005 + 0.324 = 0.3345$, so $\sigma_t \approx 0.58\%$. Volatility drifts down a little.

In the project. Not in the code yet. A GARCH VaR would replace the fixed σ of the 500-day window with a σ_t that reacts to yesterday's move.

A8. Exchange rates

2.32 Random walk

In plain words. A path where each step is random and does not depend on the earlier steps. Your best guess for tomorrow's price is simply today's price.

Tiny example. A drunk person walking along a pavement: each step is left or right at random. Where they are now is your best guess for where they will be in a minute, but the longer you wait, the further away they can be.

In the project. The 2040 scenario is essentially a random walk in log prices, with steps drawn from history. In exchange rate research, the random walk is the famous benchmark that economic models struggle to beat out of sample (Meese and Rogoff, 1983; Rossi, 2013).

3 Part B: The project step by step

This part follows `var_eurusd.py` from top to bottom.

3.1 Step 1: Data and log returns

The script downloads the ECB EUR/USD reference rate (one number per working day) and saves it in `data/eurusd.csv`. Days with an empty value are skipped, and the dates are sorted so that the oldest comes first.

- First day: 4 January 1999, rate 1.1789.
- Last day: 7 October 2026, rate 1.1177.
- 7108 daily log returns.

Example

Rates on three days: 1.1000, 1.1110, 1.1055.

- $r_2 = 100 \ln(1.1110/1.1000) = +0.995\%$

- $r_3 = 100 \ln(1.1055/1.1110) = -0.496\%$

In code: `100 * np.diff(np.log([1.1000, 1.1110, 1.1055]))`.

A detail: the return from Monday to Tuesday belongs to Tuesday. That is why the code uses `dates[1:]` for the dates of the returns.

3.2 Step 2: Are the returns normal? (Jarque–Bera)

Summary numbers for all 7108 returns:

Mean	−0.0008%
Standard deviation	0.58%
Skewness	0.05
Excess kurtosis	3.31
Jarque–Bera	3243.9
p-value	< 0.001

How to read it: skewness is near 0, so the returns are about symmetric. But the excess kurtosis of 3.31 is far from 0. Put into the formula, the Jarque–Bera statistic is about 3244, while the 5% critical value is only 5.99. **Normality is rejected.** This fits what is known about financial returns in general (Cont, 2001) and exchange rates in particular (Baillie and Bollerslev, 1989).

Two pictures show the same thing.

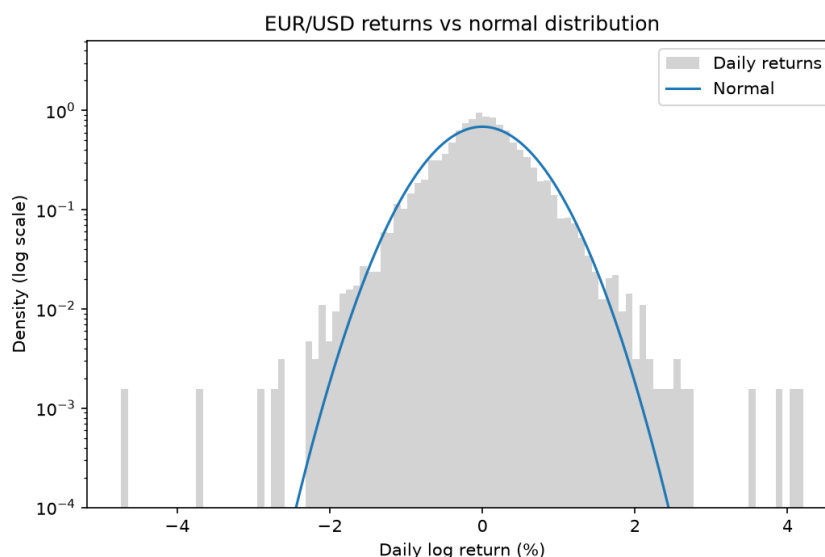


Figure 1: Histogram of daily EUR/USD log returns with a fitted normal curve. The y-axis is logarithmic so that the rare tail days are visible. In the tails, the bars stay above the normal curve: more extreme days than the normal predicts.

Common mistake

Reading the histogram on a normal (not log) y-axis and saying “it looks like a bell curve”. In the middle it does. The difference is in the tails, which are invisible on a normal axis.

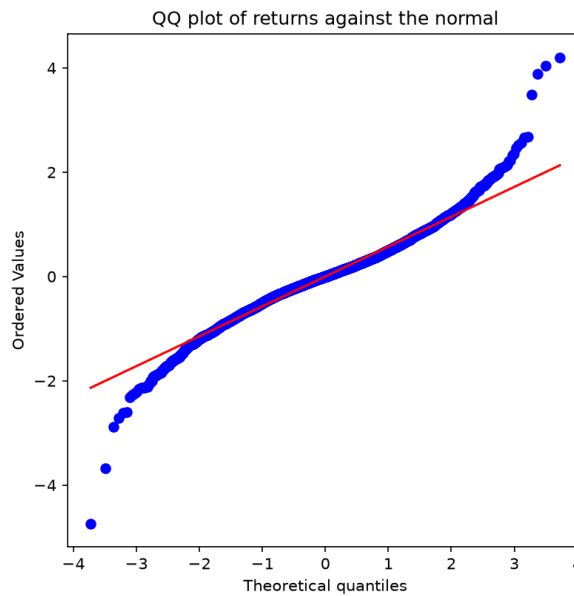


Figure 2: QQ plot of the returns against a normal distribution. The middle follows the line; both ends bend away. That S-shape means fat tails.

That is why the code uses `plt.yscale("log")`.

3.3 Step 3: Three ways to compute VaR

For each day, the script looks at the 500 returns before it and computes:

1. **Normal:** $-(\mu + z\sigma)$ with the window's μ and σ .
2. **Historical:** minus the 1% (or 5%) quantile of the 500 returns.
3. **Monte Carlo normal:** 10,000 draws from a normal with the window's μ and σ , then minus the 1% (or 5%) quantile.

All three methods are standard (Jorion, 2007).

Example: one window, two methods

Suppose a 500-day window has $\mu = 0$ and $\sigma = 0.55\%$.

- Normal 99% VaR = $2.326 \times 0.55 = 1.28\%$.
- Suppose the 5th worst return in the window is -1.45% . Then the historical 99% VaR is about 1.45%.

If the tails are fat, the historical number is larger. That is exactly the pattern in the project.

3.4 Step 4: The rolling backtest without look-ahead

The loop runs from day 500 to the last day. Every day:

1. Take `window = returns[day - 500:day]`. This stops just *before* day, so today's return is not in it.
2. Compute the six VaR numbers (3 methods $\times$ 2 levels).
3. Later, compare today's real return with each VaR.

That gives $7108 - 500 = 6608$ test days. Figure 3 shows the result for the 99% level.

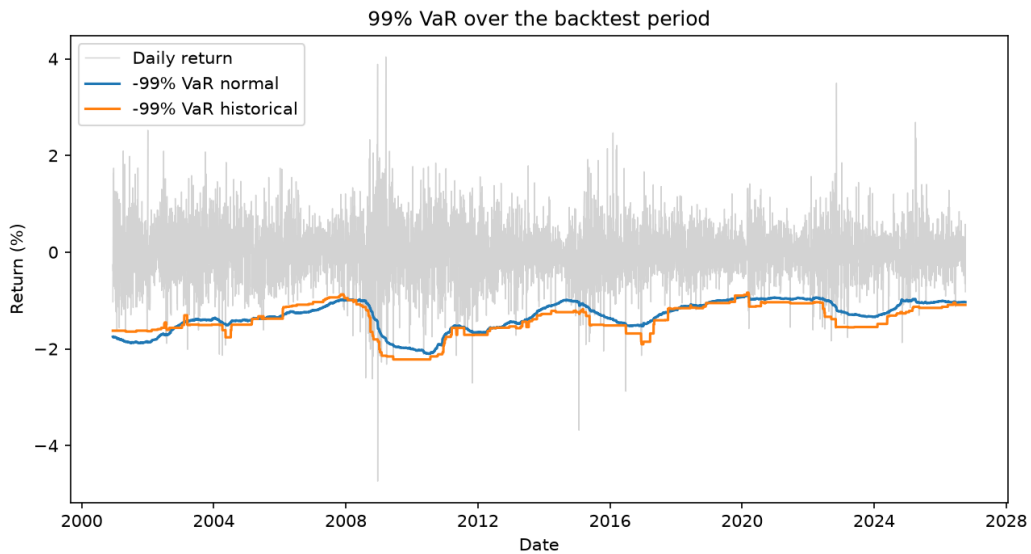


Figure 3: Daily returns (grey) and minus the 99% VaR for the normal and historical methods. Every grey spike below a line is an exceedance for that method. Note the wider grey band in 2008–2009: volatility clustering.

3.5 Step 5: Counting exceedances and the Kupiec test

These are the main results.

Method	Level	Exceedances	Expected	Kupiec p-value
Normal	95%	302	330.4	0.1040
Historical	95%	324	330.4	0.7171
Monte Carlo	95%	302	330.4	0.1040
Normal	99%	114	66.1	0.0000
Historical	99%	74	66.1	0.3367
Monte Carlo	99%	114	66.1	0.0000

Average 99% VaR over the backtest: normal 1.32%, historical 1.41%.

How to read the table, row by row.

- **95%, all methods pass.** The normal model has 302 breaks when 330 were expected; $p = 0.10$, above 0.05. So the Kupiec test does not reject the 95% VaR of the normal model. This only says the *frequency* of breaks is consistent with 5%. It does not mean that returns are normal; the Jarque–Bera test above rejects that strongly.
- **99%, normal fails.** 114 breaks instead of 66 is about *1.7 times* too many. The Kupiec statistic is about 28.8, far above 3.84, so $p < 0.001$. The normal model underestimates risk deep in the tail.
- **99%, historical passes.** 74 breaks against 66 expected, $p = 0.34$. No evidence against it.
- **Monte Carlo = normal.** Same assumption, so very close results (identical counts in this run).

In one sentence

The normal distribution is good enough in the “ordinary bad day” zone (95%), but too optimistic in the “really bad day” zone (99%), because EUR/USD has fat tails.

Example: why 99% and not 95%?

Under a normal, the 95% cutoff is 1.645σ and the 99% cutoff is 2.326σ . Fat tails mean there is extra probability far out. At 1.645σ you are not yet “far out”, so the extra probability barely shows. At 2.326σ you are, so it shows clearly.

Common mistake

Saying “the historical method is correct”. Passing Kupiec means “not rejected”, not “right”. Historical simulation has its own problems (Pritsker, 2006), and the project does not test whether its breaks come in clusters (Christoffersen, 1998).

3.6 Step 6: Scenario analysis to 2040 (bootstrap)

The last part of the script asks: “If the future days look like the past days, what range of EUR/USD rates could we see at the end of 2040?”

1. Start at the last rate, 1.1177.
2. Count the working days left until 31 December 2040 (about 3700).
3. For each of 5000 paths, draw that many daily returns at random from the 7108 historical returns, with replacement (bootstrap).
4. Add up the log returns along each path and turn them back into rates,

$$P_T = P_0 \exp(\sum r_t/100).$$

5. At each future day, take the 5th, 50th and 95th percentile over the 5000 paths.

Result at the end of 2040, *under the bootstrap model* (not a validated forecast interval for the real rate): 5th percentile about 0.62, median about 1.10, 95th percentile about 1.99.

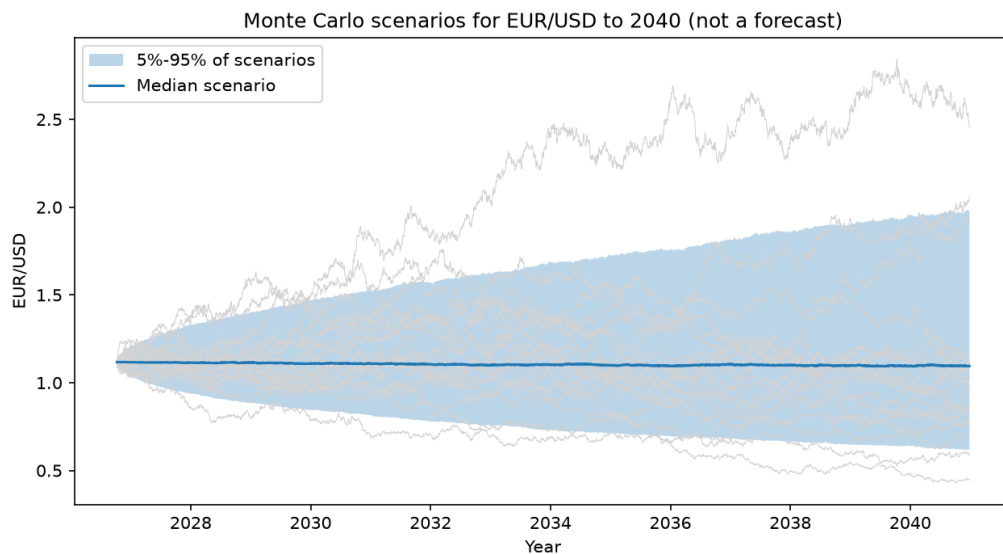


Figure 4: 5000 bootstrap scenarios for EUR/USD to the end of 2040. Grey: 20 example paths. Shaded: the 5–95% band. Line: the median. This is a scenario analysis, not a forecast.

Example: a back-of-the-envelope check

Daily $\sigma \approx 0.58\%$, about 3700 days. If days are independent, the standard deviation of the *sum* grows with the square root of time: $0.58 \times \sqrt{3700} \approx 35\%$. A 90% band is about $\pm 1.645 \times 35\% \approx \pm 58\%$ in log terms. Then $1.1177 \cdot e^{-0.58} \approx 0.63$ and $1.1177 \cdot e^{+0.58} \approx 2.00$. Very close to the simulated 0.62 and 1.99. (The small mean of -0.0008% per day adds up to about -3% over 3700 days, which pulls the median a little below the start rate.)

Why this is not a forecast (my own reasoning).

- The model has no view on direction. The average return is about zero, so the median path stays near today's rate. That is not a prediction that “the euro will stay at 1.10”; it is what happens when you have no information about direction.
- The width of the band is the message: anything from about 0.6 to about 2.0 is inside the 90% band. That is a statement about uncertainty, not about a likely outcome.
- It only re-uses 1999–2026. A new kind of event cannot appear.
- Even serious economic models struggle to forecast exchange rates better than a random walk (Meese and Rogoff, 1983; Rossi, 2013). A simple bootstrap should not claim to do better.

Common mistake

Showing the fan chart and saying “the model predicts EUR/USD at 1.10 in 2040”. The model predicts nothing. It shows how wide the range of possibilities is.

4 Part C: How currency exchange rates vary with macroeconomic trends

This part is educational. The trading story is fictive and all numbers in the orange boxes are made up. Nothing here is investment advice or a view on where EUR/USD is going.

Part B treated EUR/USD returns as random numbers. But behind the numbers are real economies: interest rates, inflation, central banks and fear. Textbooks on exchange rate economics cover all of this in depth (Sarno and Taylor, 2003). Here is a short tour, built on one fictive scenario.

Fictive story – made-up numbers

We run a small (imaginary) FX trading desk. We hold a position in EUR/USD: we own euros and count our profit in dollars. Today EUR/USD is **1.10**. Our job is to understand what could move the rate, and to be honest about how little we can predict.

4.1 Interest rate differentials and interest rate parity

Idea. Money likes to earn interest. If dollar deposits pay more than euro deposits, investors are tempted to move money into dollars.

Interest rate parity says that, in theory, you cannot get a free lunch from the interest gap. There are two versions:

- **Covered parity:** if you lock in the future exchange rate today with a forward contract, the interest gap is cancelled by the gap between the forward rate and today's rate.
- **Uncovered parity (UIP):** without a forward contract, the high-interest currency is *expected* to weaken by about the interest gap. With EUR/USD = S (dollars per euro):

$$\text{expected \% change in } S \approx i_{\text{USD}} - i_{\text{EUR}}.$$

Fictive story – made-up numbers

Dollar interest is 5% a year and euro interest is 3%. UIP says the euro should be expected to *rise* by about 2% over the year (from 1.10 to about 1.122), so that euro holders, who earn 2% less interest, are compensated by the currency gain. Then the total return is the same in both currencies.

What the data say. UIP often does not hold in the data. The forward rate turns out to be a biased guess of the future spot rate (Fama, 1984). So “high interest rate = currency will weaken” is a theory benchmark, not a trading rule.

Common mistake

Mixing up “what happens on the day rates change” and “what UIP says afterwards”. They can point in opposite directions. See overshooting below.

4.2 Inflation and purchasing power parity (PPP)

Idea. The same basket of goods should cost about the same in two places after converting the currency. If prices in the US rise faster than in the euro area, each dollar buys less, so over time

you should need more dollars per euro.

Absolute PPP says a basket costs the same in both places after conversion, $S = P_{\text{US}}/P_{\text{EA}}$.

Relative PPP is the weaker, more used version about *changes*:

$$\% \text{ change in } S \approx \pi_{\text{US}} - \pi_{\text{EA}},$$

where π is inflation.

Fictive story – made-up numbers

US inflation is 4% a year, euro area inflation is 2%. Relative PPP says EUR/USD should drift up by about 2% a year: from 1.10 to about 1.12 after one year, and further after that.

What the data say. PPP works poorly in the short run. Real exchange rates swing a lot and move back towards PPP only slowly; this is called the “PPP puzzle” (Rogoff, 1996). Whether PPP holds in the long run has been debated for a long time (Taylor and Taylor, 2004). For a daily VaR, inflation hardly matters; for a 14-year scenario to 2040 it could matter, and our bootstrap ignores it.

4.3 Central bank policy: ECB vs Fed

The European Central Bank (ECB) sets interest rates for the euro area; the Federal Reserve (Fed) sets them for the US. Their decisions change the interest rate gap from the previous section, and markets react to news about *future* decisions too, not only to the decisions themselves.

Fictive story – made-up numbers

Before a Fed meeting, everyone expects the Fed to keep its rate unchanged. Instead it raises the rate by 0.50 percentage points. The ECB does nothing. The dollar interest rate is now more attractive than expected. In our story, EUR/USD falls from 1.10 to 1.09 within minutes. Our euro position loses about 0.9%.

Two lessons from the story:

- It is the *surprise* that moves the price. A decision everyone expected is already in the price.
- One central bank meeting can produce a day in the tail of the distribution. Those days are part of why the tails are fat.

Researchers have tried to use central bank rules (“Taylor rules”, where the interest rate depends on inflation and the output gap) to predict exchange rates out of sample; one example is Molodtsova and Papell (2009). For a broad survey of what does and does not predict exchange rates, see Rossi (2013).

4.4 Dornbusch overshooting

Dornbusch (1976) explained why exchange rates can jump *more* than seems reasonable. Goods prices are sticky: they change slowly. Exchange rates are not: they change in seconds. So after a monetary policy shock, the exchange rate takes the whole adjustment at first and goes too far (overshoots), and then partly comes back as prices slowly catch up.

Fictive story – made-up numbers

(Dornbusch’s own example is a monetary expansion. Our Fed hike is the mirror image: a contractionary shock.) Continue the Fed story. After the surprise hike, EUR/USD drops from 1.10 to 1.07 (the dollar jumps up). Over the next year it drifts back up to about 1.09. Notice how this fits UIP: the dollar now pays more interest, and after the jump it is *expected to weaken* slowly, which is exactly what UIP asks for. The jump today makes room for the expected slow fall later.

In one sentence

Overshooting: the exchange rate moves too far at once because prices are slow, then partly moves back.

4.5 Safe-haven demand for the US dollar

Idea. When markets are scared, investors sell risky assets and buy what they see as safe. Some currencies tend to gain in such stress episodes; they are called safe-haven currencies (Ranaldo and Söderlind, 2010). Habib and Stracca (2012) study what makes a currency a safe haven. The US dollar has a special role as the world’s main reserve currency (Maggiori, 2017).

Fictive story – made-up numbers

A sudden global financial panic starts. In our story, investors around the world buy US dollars and US government bonds. EUR/USD falls from 1.10 to 1.05 in two weeks, even though nothing changed in euro area interest rates or inflation. Our euro position loses about 4.7%. The fall had nothing to do with “fundamentals” in the textbook sense. It came from fear.

This is also a link back to Part B: stress periods are where volatility clusters and where fat-tail days live. A VaR model built on calm years will be surprised by them.

4.6 The honest message: models struggle to beat a random walk

After all this theory, the most important result in the field is a humble one. Meese and Rogoff (1983) compared economic exchange rate models with a simple random walk (“tomorrow = today”) and found that the models did not beat it out of sample. Later work shows that this is still a hard benchmark to beat, although predictability has been found for some models, samples and forecast horizons (Rossi, 2013; Molodtsova and Papell, 2009). Engel and West (2005) give one explanation: even if exchange rates are driven by fundamentals like interest rates and money supply, they can still look almost like a random walk.

Fictive story – made-up numbers

Our trading desk builds a beautiful model with interest rates, inflation and central bank signals. On past data it “explains” a lot. But when we test it on new data, it guesses tomorrow’s EUR/USD no better than just using today’s rate. This can well happen: the random walk is a very hard benchmark to beat.

What this means for the project (my own reasoning):

- It is reasonable that the project models *risk* (how big the moves can be) and not *direction* (which way they go). Risk is the part we can measure with some confidence.
- It is also why the 2040 fan chart is a scenario analysis: if expert models cannot reliably beat “tomorrow = today”, a bootstrap of past returns should not pretend to know 2040.

In one sentence

Macro forces clearly move exchange rates, but turning them into reliable forecasts is very hard, so the project measures risk instead of predicting direction.

5 Limitations

Every project has limits. Saying them clearly is part of the result.

1. **Independent days in the bootstrap.** Every future day is drawn on its own. In real data, calm days cluster and wild days cluster. The bootstrap mixes a 2008 day next to a quiet 2014 day, so long stormy spells are probably too rare in the scenarios.
2. **Volatility clustering is ignored in the VaR.** The 500-day window gives equal weight to a day two years ago and yesterday. After a sudden shock the VaR reacts slowly. A GARCH model (Bollerslev, 1986) would react faster. The Kupiec test also does not check if exceedances cluster (Christoffersen, 1998).
3. **Only 1999–2026 history.** The bootstrap and the historical VaR can only produce moves that happened in that period. A shock bigger than the worst day so far is impossible in the model.
4. **One currency pair.** EUR/USD is the most traded pair and may behave differently from smaller currencies. Results for EUR/SEK or EUR/CHF could be different.
5. **No costs.** No bid/ask spread, no trading fees, no interest from holding the currencies. A real position would earn or pay interest every day.
6. **One snapshot per day.** The ECB rate is a snapshot in the early afternoon. Moves inside the day are invisible.
7. **Calendar detail.** The scenario counts 261 weekdays per year, a little more than the number of ECB working days, so the 2040 band is very slightly wider than it would otherwise be.

6 Exercises

Try them before you look at the answers.

1. Prices are 1.20, 1.26, 1.197. Compute the two daily log returns in percent and their sum. What is $100 \ln(1.197/1.20)$?
2. EUR/USD goes from 1.15 to 1.13. Did the euro or the dollar get stronger? Is the return positive or negative?
3. SciPy says excess kurtosis = 0.1. Is the distribution’s kurtosis close to that of a normal?

4. $n = 1200$, $S = 0.2$, $K = 0.5$. Compute the Jarque–Bera statistic. Is normality rejected at 5% (critical value 5.99)?
5. A window has $\mu = 0.01\%$ and $\sigma = 0.70\%$. Compute the 95% and the 99% normal VaR.
6. A 99% VaR is tested on 2000 days. How many exceedances do you expect? You see 40. Is that likely to be a problem?
7. A friend writes `window = returns[day - 500:day + 1]`. What is wrong?
8. Why does the Monte Carlo VaR in the project give the same number of exceedances as the normal VaR?
9. Name two reasons why the 2040 fan chart is not a forecast.
10. In the fictive story, US inflation is 3% and euro area inflation is 3%. What does relative PPP say about EUR/USD?

Short answers

1. $100 \ln(1.26/1.20) = +4.88\%$, $100 \ln(1.197/1.26) = -5.13\%$. Sum = -0.25% , which equals $100 \ln(1.197/1.20) \approx -0.25\%$. Log returns add up.
2. The rate fell, so one euro buys fewer dollars: the dollar got stronger, the euro weaker. The return is negative: $100 \ln(1.13/1.15) \approx -1.75\%$.
3. Yes. Kurtosis = $3 + 0.1 = 3.1$, close to the normal's 3.
4. $JB = \frac{1200}{6}(0.04 + 0.25/4) = 200 \times 0.1025 = 20.5 > 5.99$. Rejected.
5. 95%: $-(0.01 - 1.645 \times 0.70) = 1.14\%$. 99%: $-(0.01 - 2.326 \times 0.70) = 1.62\%$.
6. Expected $0.01 \times 2000 = 20$. Seeing 40 is double. The Kupiec test rejects clearly (LR is about 15, far above 3.84). The VaR is too low.
7. The slice now includes `returns[day]`, the return being tested. That is look-ahead bias.
8. It draws from a normal distribution with the same μ and σ . With 10,000 draws the simulated quantile is almost the formula quantile. Same assumption, same thin tails.
9. The model has no view on direction (mean about zero); it only re-uses the past; and exchange rates are known to be very hard to forecast (Meese and Rogoff, 1983; Rossi, 2013).
10. The inflation gap is zero, so PPP gives no reason for the rate to drift in either direction.

7 Cheat sheet

What	Formula / rule
EUR/USD quote	1 euro costs S dollars. S up = euro stronger = positive return.
Log return (%)	$r_t = 100 \ln(P_t/P_{t-1})$
Back to a price	$P_T = P_0 \exp(\sum_t r_t/100)$
Mean	$\mu = \frac{1}{n} \sum r_t$
Standard deviation	$\sigma = \sqrt{\frac{1}{n} \sum (r_t - \mu)^2}$
Square-root-of-time	$\sigma_{T \text{ days}} \approx \sigma_{1 \text{ day}} \sqrt{T}$ (independent days)
Excess kurtosis	kurtosis $- 3$ (normal: 0)
Jarque-Bera	$JB = \frac{n}{6} \left(S^2 + \frac{K^2}{4} \right) \sim \chi^2(2)$; 5% critical value 5.99
Normal VaR	$\text{VaR} = -(\mu + z\sigma)$, $z = \Phi^{-1}(1 - \text{level})$
z values	95%: -1.645 99%: -2.326
Historical VaR	$-\text{quantile}_{1-\text{level}}$ of past returns
Exceedance	$r_t < -\text{VaR}_t$
Expected exceedances	$(1 - \text{level}) \times n$
Kupiec LR	$-2 \ln \left[\frac{(1-p)^{n-x} p^x}{(1-\frac{x}{n})^{n-x} (\frac{x}{n})^x} \right] \sim \chi^2(1)$; 5% critical value 3.84
ARCH(1)	$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2$
GARCH(1,1)	$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2$
UIP	expected % change in $S \approx i_{\text{USD}} - i_{\text{EUR}}$
Relative PPP	% change in $S \approx \pi_{\text{US}} - \pi_{\text{EA}}$
p-value rule	$p < 0.05$: reject H_0 . $p \geq 0.05$: no evidence against H_0 (not proof).
Project results	JB = 3243.9; 99%: normal 114 vs 66.1 (rejected), historical 74 ($p = 0.34$); 95%: both pass.

References

- Philippe Artzner, Freddy Delbaen, Jean-Marc Eber, and David Heath. Coherent measures of risk. *Mathematical Finance*, 9(3):203–228, 1999. doi: 10.1111/1467-9965.00068.
- Richard T. Baillie and Tim Bollerslev. The message in daily exchange rates: A conditional-variance tale. *Journal of Business & Economic Statistics*, 7(3):297–305, 1989. doi: 10.1080/07350015.1989.10509739.
- Basel Committee on Banking Supervision. Supervisory framework for the use of “backtesting” in conjunction with the internal models approach to market risk capital requirements. Technical report, Bank for International Settlements, Basel, January 1996. URL <https://www.bis.org/publ/bcbs22.htm>. Historical document; marked as superseded on the BIS website.
- Tim Bollerslev. Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3):307–327, 1986. doi: 10.1016/0304-4076(86)90063-1.
- Tim Bollerslev, Ray Y. Chou, and Kenneth F. Kroner. ARCH modeling in finance. *Journal of Econometrics*, 52(1–2):5–59, 1992. doi: 10.1016/0304-4076(92)90064-X.
- Peter F. Christoffersen. Evaluating interval forecasts. *International Economic Review*, 39(4):841–862, 1998. doi: 10.2307/2527341.
- Rama Cont. Empirical properties of asset returns: Stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236, 2001. doi: 10.1080/713665670.
- Rudiger Dornbusch. Expectations and exchange rate dynamics. *Journal of Political Economy*, 84(6):1161–1176, 1976. doi: 10.1086/260506.
- B. Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1), 1979. doi: 10.1214/aos/1176344552.
- Bradley Efron and Robert Tibshirani. Bootstrap methods for standard errors, confidence intervals, and other measures of statistical accuracy. *Statistical Science*, 1(1), 1986. doi: 10.1214/ss/1177013815.
- Charles Engel and Kenneth D. West. Exchange rates and fundamentals. *Journal of Political Economy*, 113(3):485–517, 2005. doi: 10.1086/429137.
- Robert F. Engle. Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4):987–1007, 1982. doi: 10.2307/1912773.
- European Central Bank. Euro foreign exchange reference rates. ECB website, 2026. URL https://www.ecb.europa.eu/stats/policy_and_exchange_rates/euro_reference_exchange_rates/html/index.en.html. Accessed 2026-10-07.
- Eugene F. Fama. The behavior of stock-market prices. *The Journal of Business*, 38(1):34, 1965. doi: 10.1086/294743.
- Eugene F. Fama. Forward and spot exchange rates. *Journal of Monetary Economics*, 14(3):319–338, 1984. doi: 10.1016/0304-3932(84)90046-1.
- Paul Glasserman. *Monte Carlo Methods in Financial Engineering*. Stochastic Modelling and Applied Probability. Springer, New York, 2003. doi: 10.1007/978-0-387-21617-1.
- Maurizio Michael Habib and Livio Stracca. Getting beyond carry trade: What makes a safe haven currency? *Journal of International Economics*, 87(1):50–64, 2012. doi: 10.1016/j.jinteco.2011.12.005.

- Peter R. Hansen and Asger Lunde. A forecast comparison of volatility models: Does anything beat a GARCH(1,1)? *Journal of Applied Econometrics*, 20(7):873–889, 2005. doi: 10.1002/jae.800.
- Carlos M. Jarque and Anil K. Bera. Efficient tests for normality, homoscedasticity and serial independence of regression residuals. *Economics Letters*, 6(3):255–259, 1980. doi: 10.1016/0165-1765(80)90024-5.
- Carlos M. Jarque and Anil K. Bera. A test for normality of observations and regression residuals. *International Statistical Review*, 55(2):163–172, 1987. doi: 10.2307/1403192.
- Philippe Jorion. *Value at Risk: The New Benchmark for Managing Financial Risk*. McGraw-Hill, 3rd edition, 2007. ISBN 978-0-07-146495-6.
- J.P. Morgan/Reuters. RiskMetrics — technical document. Technical report, J.P. Morgan and Reuters, New York, 1996. URL <https://www.msci.com/research-and-insights/paper/1996-riskmetrics-technical-document>.
- Paul H. Kupiec. Techniques for verifying the accuracy of risk measurement models. *The Journal of Derivatives*, 3(2):73–84, 1995. doi: 10.3905/jod.1995.407942.
- Matteo Maggiori. Financial intermediation, international risk sharing, and reserve currencies. *American Economic Review*, 107(10):3038–3071, 2017. doi: 10.1257/aer.20130479.
- Benoit Mandelbrot. The variation of certain speculative prices. *The Journal of Business*, 36(4):394, 1963. doi: 10.1086/294632.
- Richard A. Meese and Kenneth Rogoff. Empirical exchange rate models of the seventies: Do they fit out of sample? *Journal of International Economics*, 14(1–2):3–24, 1983. doi: 10.1016/0022-1996(83)90017-X.
- Tanya Molodtsova and David H. Papell. Out-of-sample exchange rate predictability with Taylor rule fundamentals. *Journal of International Economics*, 77(2):167–180, 2009. doi: 10.1016/j.jinteco.2008.11.001.
- Matthew Pritsker. The hidden dangers of historical simulation. *Journal of Banking & Finance*, 30(2):561–582, 2006. doi: 10.1016/j.jbankfin.2005.04.013.
- Angelo Ranaldo and Paul Söderlind. Safe haven currencies. *Review of Finance*, 14(3):385–407, 2010. doi: 10.1093/rof/rfq007.
- Kenneth Rogoff. The purchasing power parity puzzle. *Journal of Economic Literature*, 34(2):647–668, 1996. URL <https://ideas.repec.org/a/aea/jecclit/v34y1996i2p647-668.html>.
- Barbara Rossi. Exchange rate predictability. *Journal of Economic Literature*, 51(4):1063–1119, 2013. doi: 10.1257/jel.51.4.1063.
- Lucio Sarno and Mark P. Taylor. *The Economics of Exchange Rates*. Cambridge University Press, 2003. doi: 10.1017/cbo9780511754159.
- Alan M. Taylor and Mark P. Taylor. The purchasing power parity debate. *Journal of Economic Perspectives*, 18(4):135–158, 2004. doi: 10.1257/0895330042632744.